2-DAY INSTRUCTOR-LED CORPORATE TRAINING

Microsoft Power Apps for End Users

Build Business Apps Without Coding

From Manual Processes to Smart Business Applications

Duration: 2 Days | Level: Beginner to Intermediate | No programming experience required





Fakhrul Syahmi

Data Scientist &
Data Analytics Consultant

About the Trainer

- **Artificial Intelligence & Machine Learning**
- **Microsoft Power Platform Expert**
- **Data Analytics & Business Intelligence**
- **Process Automation & Digital Transformation**
- **Copilot Studio & AI Agent Development**
- **Certified Microsoft Technology Trainer**

Passionate about empowering organisations with AI-driven solutions and helping professionals unlock the potential of Microsoft technologies.

Who this course is for

Designed for business people who solve business problems - not for professional developers.

Business users
Anyone running a manual or spreadsheet-based process

Executives & managers
Sponsors who need visibility and faster approvals

Administrative staff
Form, request and tracking workloads

HR teams
Leave, onboarding, training requests

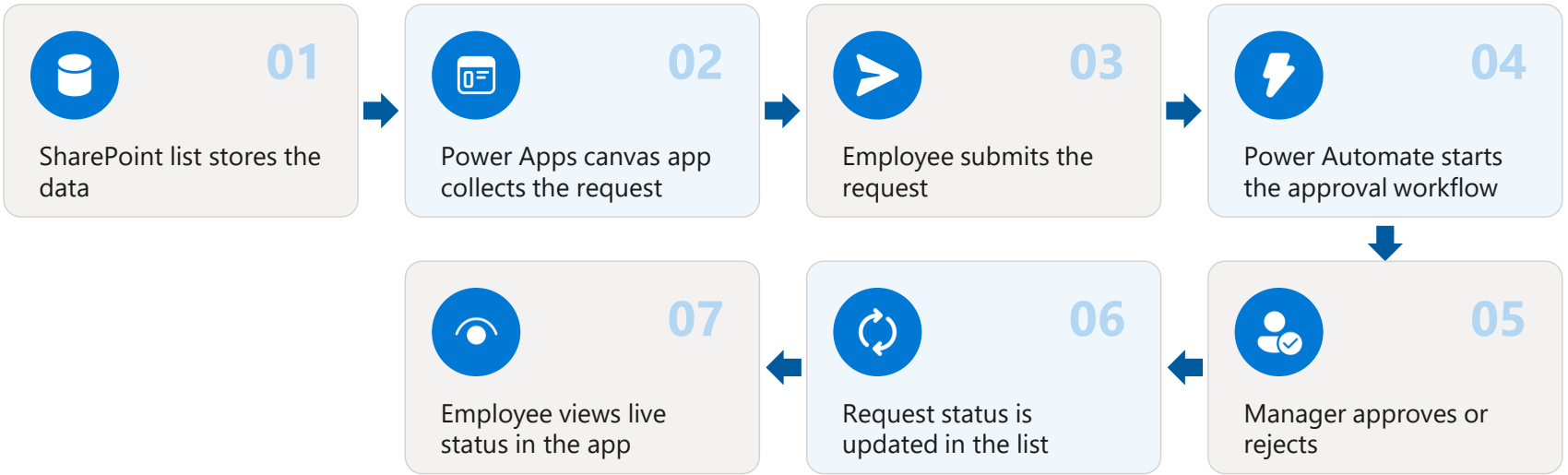
Finance teams
Purchase, claims and budget approvals

Operations & projects
Checklists, inspections, task tracking

Prerequisite: comfort with Microsoft 365. Programming experience is not required.

What you will build over two days

One project, built progressively: an Employee Request & Approval App.



30% concept, 70% hands-on



1. Concept

A short, plain-language explanation of the idea - no jargon.



2. Business example

How a real HR, finance or operations team uses it.



3. Demonstration

The trainer builds it live so you see every click.



4. Hands-on

You build the same thing in your own environment.



5. Key takeaway

One sentence you can repeat back to your team.

Slide labels tell you what to expect: CONCEPT, DEMO, HANDS-ON, EXERCISE, TIP, BEST PRACTICE, CAPSTONE.

Two days at a glance

Day 1 - Build your first canvas app

- Power Platform and Power Apps basics
- Canvas vs model-driven apps
- Power Apps Studio tour
- Data sources and Microsoft Lists
- Create the Employee Requests list
- First canvas app and modern controls
- Screens, navigation and forms
- Galleries and interface design

Day 2 - Logic, automation and delivery

- Power Fx essentials
- Variables and collections
- Patch and validation
- Search, filtering and status
- Power Automate approvals
- End-to-end testing
- Sharing, permissions and mobile
- AI features, code view, capstone

DAY 1

Building Your First Power Apps Canvas Application

Modules 1-12

Platform | Studio | Data | Screens | Forms | Galleries

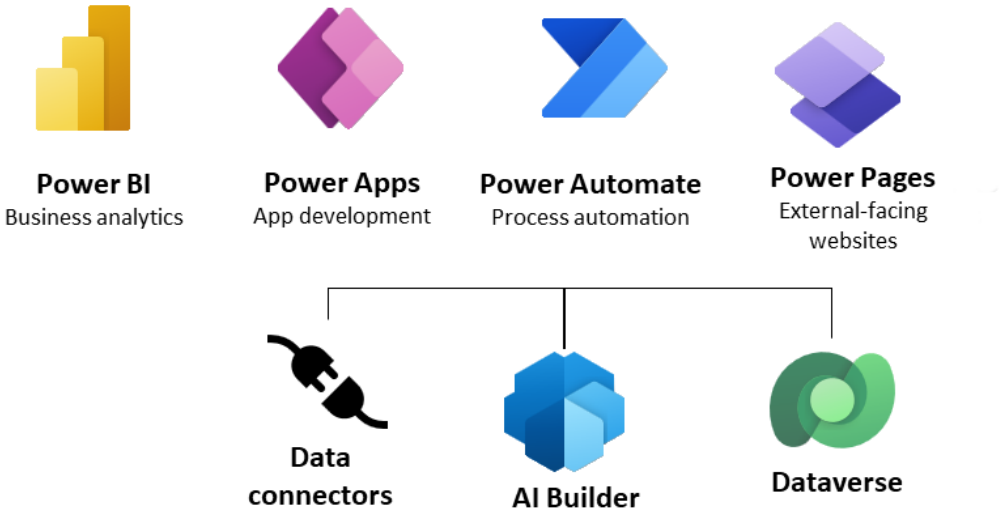
MODULES 1-3

Foundations: Platform, Power Apps, App Types

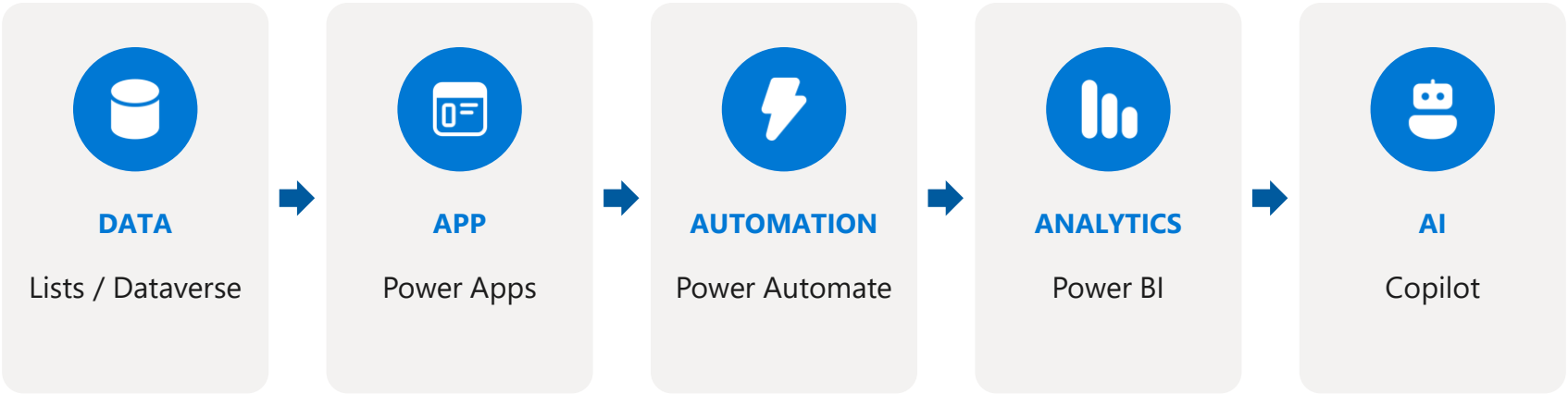
Understand the ecosystem before you build anything

What is Microsoft Power Platform?

The low code platform that spans Microsoft 365, Azure, Dynamics 365, and standalone apps.



How the pieces fit together



Everything starts with data. The app is how people interact with it, automation moves it forward, analytics explains it, and AI accelerates all of it.

What each component looks like in real work

Power Apps

Mobile site-inspection form replacing paper checklists

Operations

Power Automate

Purchase requests routed to the right approver automatically

Finance

Power BI

Dashboard showing approval turnaround time by department

Management

Copilot Studio

Agent answering 'what is my leave balance?' in Teams

HR

Dataverse

Structured employee data shared across several apps

IT and HR

Microsoft 365

Teams, Outlook and SharePoint as the everyday surface

Everyone

Power Platform works inside Microsoft 365

Where your app lives

- Microsoft Teams - app embedded as a tab
- SharePoint - list, documents and site pages
- Outlook - approval emails and notifications
- Power Apps Mobile - phone and tablet access
- Browser - full desktop experience

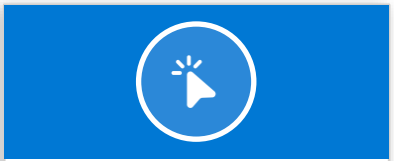


Why this matters

- No new system for people to learn or log into
- Existing Microsoft 365 sign-in and security apply
- Data stays inside your tenant
- Approvals arrive where people already work: Outlook and Teams
- IT keeps governance and visibility

What is Microsoft Power Apps?

A low-code service for building business applications using drag-and-drop design and Excel-like formulas.



Drag and drop

You design screens visually, the way you build a slide - no code editor required.



Excel-like formulas

Logic is written in Power Fx, which looks and behaves like Excel formulas.



Connected data

The app reads and writes to Lists, SharePoint, Dataverse, SQL and more.



Runs anywhere

The same app works in a browser, in Teams and on a phone.

Why organisations use Power Apps

Speed

Working app in days, not months of IT backlog

Lower cost

Reuses Microsoft 365 investment and skills

Ownership

The team that knows the process builds the app

Consistency

One version of the form and the data

Visibility

Status and history instead of lost email threads

Compliance

Audit trail, permissions and retention built in

Typical result: a request process that took 5 days by email completes in under 1 day with automated approval.

Low-code vs traditional development

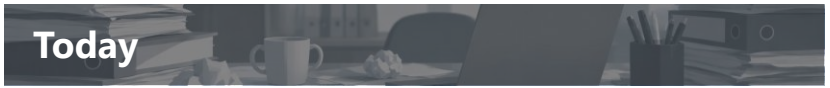
Traditional development

- Requirements document, then a long build
- Specialist developers and environments
- Weeks or months to first working version
- Changes go through a release cycle
- Business waits for IT capacity

Low-code with Power Apps

- Build a working version in the first workshop
- Business user builds, IT governs
- Hours or days to first working version
- Change a screen and republish in minutes
- IT focuses on data, security and standards

Before and after: the request process



Today

- Paper form
- Excel file on a shared drive
- Email approval
- Manual status chasing

With Power Apps

- Power Apps form
- Centralised list data
- Automated approval
- Live status for everyone

Same process, same people - the difference is where the data lives and who chases it.

Common use cases by department

HR
Leave requests, onboarding checklists, training nominations

Finance
Purchase requests, expense claims, budget approvals

Operations
Site inspections, incident logs, daily checklists

Procurement
Vendor requests, quotation tracking, goods received

Project teams
Task tracking, risk logs, status updates

Administration
Equipment bookings, visitor logs, service requests

TIP: Start with a process that has one form, one approver and a clear status - exactly like our capstone.

The three kinds of Power Apps



Canvas apps

You control the layout pixel by pixel. Best for custom interfaces, mobile apps, business forms and departmental solutions. This is our focus.



Model-driven apps

The layout is generated from your Dataverse data model. Best for structured, process-heavy enterprise applications.



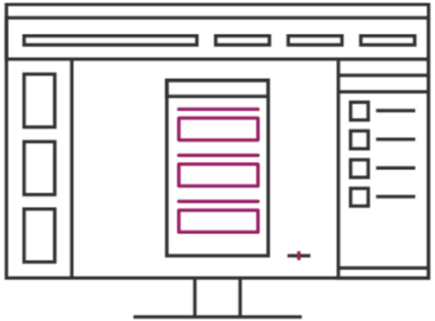
Power Pages

External-facing websites backed by Dataverse - for customers, suppliers or the public. Awareness only in this course.

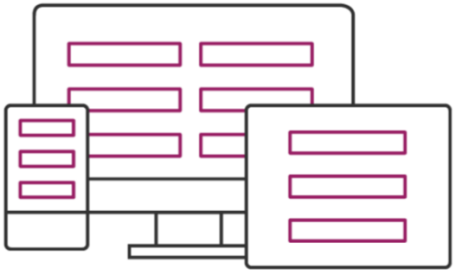
Power Apps Workflow



Connect to data & systems you're already using *easily*



Create apps, forms, and workflows *without writing code*



Use apps *on any device* – both web and mobile

Canvas vs model-driven: how to choose

	Canvas app	Model-driven app
Design control	Full control of layout and look	Layout generated by the platform
Typical data	Lists, SharePoint, Excel, SQL, Dataverse	Dataverse only
Best for	Forms, mobile apps, departmental tools	Structured enterprise processes
Skill needed	Business user with Excel-level formulas	Data modelling and configuration
Mobile	Designed for phone and tablet layouts	Responsive by default
Our course	Primary focus - the capstone app	Awareness only

Why we focus on canvas apps

- They start from a blank screen - you decide exactly what the user sees.
- They connect directly to Microsoft Lists and SharePoint, which most teams already have.
- They are the fastest route from a paper form to a working mobile app.
- They need no additional licensing in most Microsoft 365 plans for standard connectors.
- The skills transfer: navigation, forms, galleries and Power Fx appear everywhere in the platform.

Key takeaway: canvas apps are the business user's entry point to the Power Platform.

MODULES 4-6

Power Apps Studio and Your Data Source

Learn the workspace, then build the list your app will use

The Power Apps home experience

make.powerapps.com

- Home - start here, recent apps and templates
- Create - blank app, start from data, start from a description
- Apps - everything you own or that is shared with you
- Tables / Dataverse - structured data
- Flows - your Power Automate workflows
- Solutions - packaging apps and flows together

Trainer demo

- Sign in and confirm the environment name in the top right
- Show recent apps and the Create page side by side
- Point out where a shared app appears for a normal user
- Explain that Solutions matter later, for moving apps between environments

Inside Power Apps Studio

Formula bar - where every rule and behaviour is written

Tree view

Every screen and control, in order

Insert panel

Add controls, media and forms

Canvas

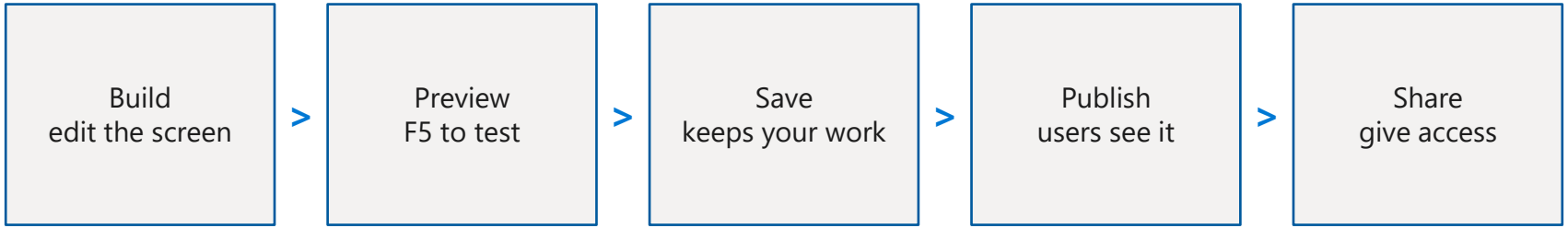
Your screen, exactly as users see it

Properties

Colour, size, text and behaviour

Select a control, choose a property, then write the value or formula. That single habit is 90% of app building.

Preview, save, publish



Saving does not release your changes. Users continue to see the last published version until you publish again.

TIP: Preview often. Save at every milestone. Publish only when the change is ready for real users.

Where your app keeps its data

Microsoft Lists / SharePoint

The default choice for departmental apps

Excel in OneDrive

Quick starts and prototypes only

Dataverse

Structured, secure, enterprise-grade data

SQL Server

Existing business databases

Teams and Outlook

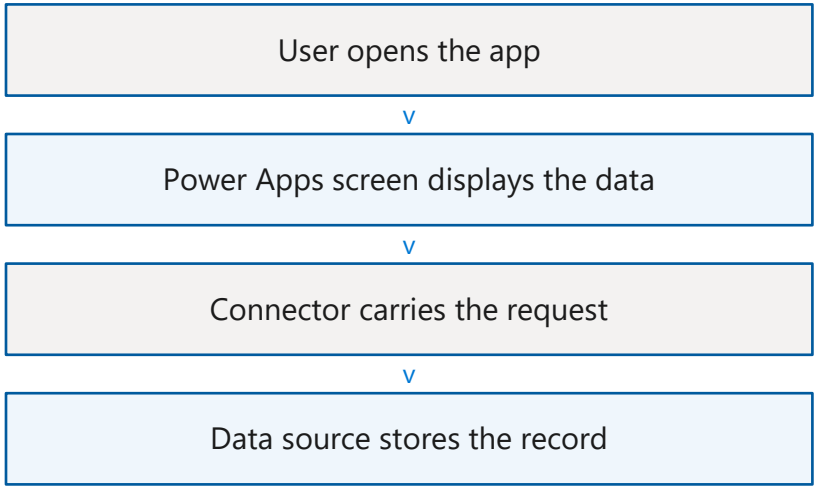
Post messages, send mail, read calendars

Other connectors

Hundreds of Microsoft and third-party services

In this course we use Microsoft Lists - available to every Microsoft 365 user and ideal for request processes.

The app is not the database



What this means in practice

- Deleting a screen never deletes your data
- The same list can feed several apps and reports
- Data lives on after the app is retired
- Permissions on the list are separate from access to the app
- Design your data first, your screens second

When Microsoft Lists beats Excel

Excel file

- One person edits comfortably at a time
- No per-item permissions
- Formatting and data mixed together
- Version chaos across copies
- Hard to automate reliably

Microsoft Lists / SharePoint

- Many people work at the same time
- Item-level permissions and audit history
- Real column types: choice, person, date
- Single source of truth
- Native trigger for Power Automate

Best practice: if more than one person updates it, or it drives an approval, it belongs in a list.

Create the Employee Requests list

1. Open Microsoft Lists or your SharePoint team site.
2. Choose New list, then Blank list.
3. Name it Employee Requests and add a short description.
4. Add the columns on the next slide, one at a time, choosing the correct type.
5. Add two sample records so your app has something to display.
6. Copy the site URL - you will need it when you connect the app.

Checkpoint

- List named exactly Employee Requests
- All columns created with correct types
- Two sample rows visible
- Site URL saved somewhere handy

Recommended column structure

Column	Type
RequestID / Title	Text
EmployeeName	Text or Person
Department	Choice
RequestType	Choice
RequestDate	Date

Column	Type
Description	Multi-line text
Amount	Number
Manager	Person
Status	Choice
ManagerComments	Multi-line text

Choice values must match your formulas and flow exactly. CreatedBy and CreatedDate already exist - do not recreate them.

Choosing the right column type

Text

Short free-form entries such as a name or reference

Number

Amounts you may total, compare or filter

Choice

A fixed set of options - keeps data consistent

Date

Real dates you can sort and calculate with

Person

A real Microsoft 365 user, ready for approvals

Yes / No

Simple flags such as Urgent or Confirmed

BEST PRACTICE: never store a status, department or approver as free text - use Choice or Person.

MODULES 7-9

Building the App: Screens, Controls, Navigation

From an empty canvas to a navigable application

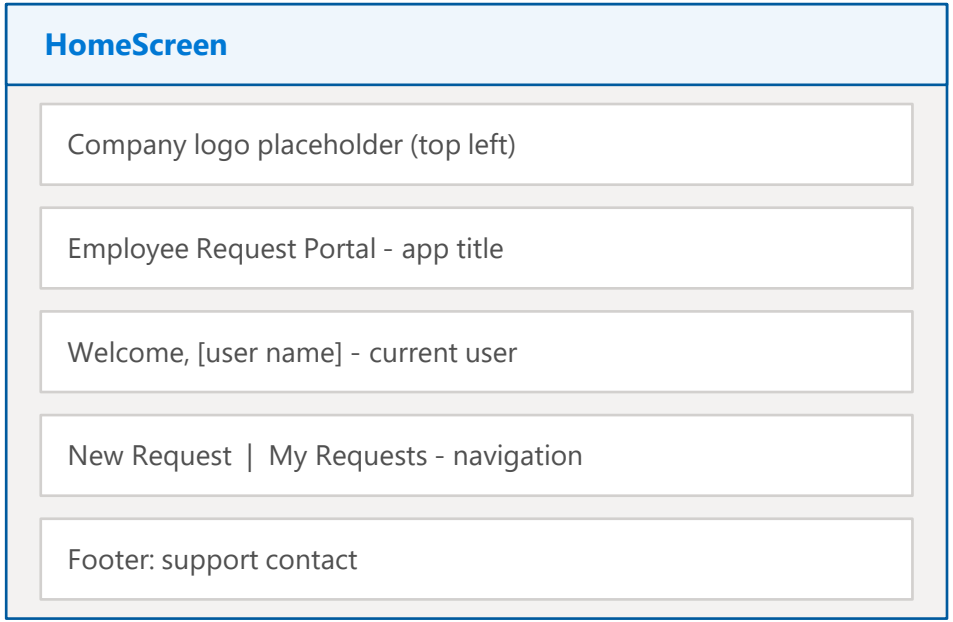
Create your first canvas app

1. In Power Apps, choose Create, then Blank app, then Blank canvas app.
2. Name it Employee Requests App and pick the Tablet layout.
3. Open the Data panel and choose Add data.
4. Select SharePoint, enter your site URL, then tick Employee Requests.
5. Save the app - this is your Day 1 foundation.

Layout choices

- Tablet - wide screens, best for learning
- Phone - tall, single-column, field work
- Responsive - adapts, needs containers
- You can change your mind later, but redesign costs time

Design the Home screen



Build these controls

- Header (modern control) for the title bar
- Image control as the logo placeholder
- Text label for the welcome message
- Two Button controls for navigation
- Keep 0.25 inch of space between elements

Your first formulas: knowing who is using the app

```
User().FullName    // the signed-in person's display name
User().Email       // their work email address

"Welcome, " & User().FullName
```

A formula is simply an instruction that produces a value - exactly like a cell formula in Excel. The & symbol joins pieces of text together.

Set the welcome label's Text property to the last formula, then preview to see your own name appear.

Modern controls: your building blocks

Button
Runs an action when tapped

Text label
Displays information

Text input
Collects typed entries

Dropdown
One option from a short list

Combo box
Searchable, supports people

Date picker
Calendar selection

Toggle
Yes or no switches

Icon and Image
Visual cues and logos

Header
Consistent screen title bar

Table
Rows of records at a glance

Form
Create and edit records

Container
Groups and aligns controls

Use modern controls throughout - they match current Microsoft 365 styling and behave better on mobile.

Control, property, value, formula

CONTROL
what you selected

>

PROPERTY
what you are setting

>

VALUE or FORMULA
what it becomes

```
Button1.Text     = "Submit Request"
Button1.OnSelect = Navigate(NewRequestScreen)
Label1.Visible   = false
```

Everything you will ever do in Power Apps follows this one pattern.

The four screens of our app

Home

Welcome, branding and navigation to everything else.

New Request

The form an employee fills in and submits.

My Requests

A gallery of the requests this person has raised.

Request Details

Full information, status and manager comments.

Moving between screens

```
// On a button: go to another screen
Navigate(NewRequestScreen)

// With a visual transition
Navigate(MyRequestsScreen, ScreenTransition.Fade)

// On a back arrow: return to the previous screen
Back()
```

EXERCISE: add a navigation button for each screen, and a Back icon on every screen except Home.

MODULES 10-12

Forms, Galleries and User Experience

Capture data, display it, and make it look professional

Display forms and edit forms

Display form

- Read-only view of one record
- Used on the Request Details screen
- Nothing can be changed by the user
- Simple and safe to place anywhere

Edit form

- Creates a new record or edits an existing one
- Used on the New Request screen
- Handles validation and saving for you
- Works with SubmitForm to write data

One form control shows one record. Galleries show many. That distinction drives every screen you design.

The four properties that control a form

DataSource

Which list the form reads and writes: 'Employee Requests'

Item

Which record to display - usually the gallery's selected item

DefaultMode

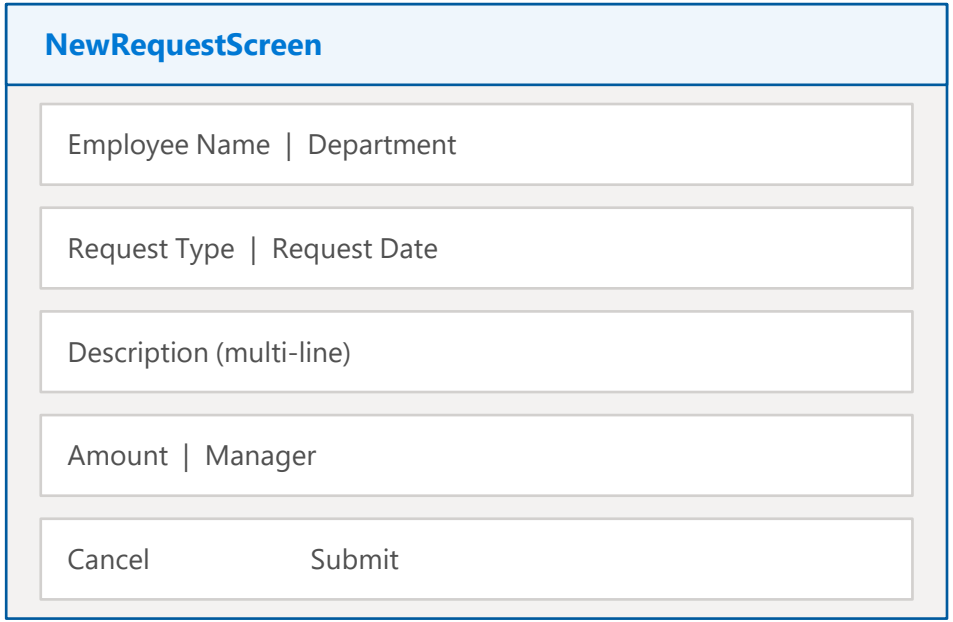
New, Edit or View - decides how the form behaves

OnSuccess

What happens after a successful save, such as navigating back

```
FormMode.New      FormMode.Edit      FormMode.View
```

Build the New Request form



Steps

- Insert an Edit form and set DataSource to Employee Requests
- Set DefaultMode to FormMode.New
- Use Edit fields to remove Status and ManagerComments
- Arrange fields in two columns for a tidy layout
- Add Submit and Cancel buttons below the form

Saving the request

```
// Submit button
SubmitForm(frmRequest)

// Clear the form ready for a new entry
NewForm(frmRequest)

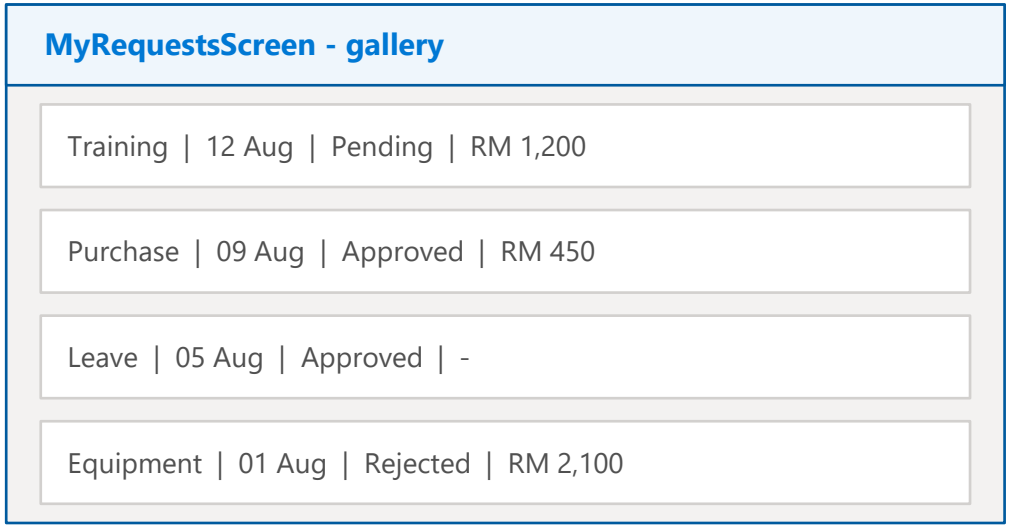
// Discard changes the user made
ResetForm(frmRequest)

// Form OnSuccess: confirm and return home
Notify("Request submitted", NotificationType.Success); Back()
```

HANDS-ON: submit a real request, then confirm the new row appears in your list.

What is a gallery?

A gallery repeats one layout for every record in your data - like a mail merge for screens.



Design one row, get them all

- You format a single template row
- Power Apps repeats it for every record
- ThisItem refers to the current row
- Selecting a row sets Gallery.Selected

Show only my requests

```
// Gallery Items property
Filter(
    'Employee Requests',
    CreatedBy.Email = User().Email
)
```

Read it as a sentence: from the Employee Requests list, keep only the rows created by the person using the app right now.

Then sort newest first: Sort(Filter('Employee Requests', CreatedBy.Email = User().Email), Created, SortOrder.Descending)

Open a request from the gallery

```
// Gallery item OnSelect
Set(varSelectedRequest, ThisItem); Navigate(RequestDetailsScreen)

// Display form on the details screen
frmDetails.Item = varSelectedRequest
```

EXERCISE: tap three different requests and confirm the details screen shows the right record each time.

Nine habits that make an app look professional

- **Consistent colours**
- **Consistent button styles**
- **Clear navigation**
- **Generous white space**
- **Proper alignment**
- **Mobile-friendly sizing**
- **Visual hierarchy**
- **Clear status indicators**
- **Accessible contrast**

Pick two accent colours and stay with them. Restraint reads as premium; variety reads as unfinished.

Containers and responsive layout

Without containers

- Every control positioned by hand
- Alignment drifts as you edit
- Nothing adapts to screen size
- Moving a section means moving ten controls

With containers

- Controls grouped in a logical block
- Horizontal and vertical auto-layout
- Sections stretch to fit the screen
- Move or hide a whole section at once

HANDS-ON: wrap your home screen buttons in a horizontal container and see them align themselves.

Polish the Employee Request App

- Apply one heading style and one body style across all four screens.
- Make every primary button the same size, colour and corner radius.
- Align controls to a common left edge and equalise the spacing.
- Add a header to each screen so users always know where they are.
- Rename every control meaningfully: btnSubmit, galRequests, frmRequest.
- Preview on the phone shape and fix anything that overflows.

End of Day 1: you have a working app that stores real requests.

DAY 2

Business Logic, Automation and Delivery

Modules 13-27

Power Fx | Variables | Patch | Validation | Automation | Sharing | AI

What we built yesterday

Canvas apps

Blank screen, full design control

Data sources

Employee Requests list in SharePoint

Screens

Home, New Request, My Requests, Details

Modern controls

Buttons, inputs, pickers, containers

Forms

Edit form plus SubmitForm to save

Galleries

Filter and Sort to show my requests

Today we add the intelligence: logic, validation, automation and delivery.

Quick quiz - five questions

1. Which control shows many records at once: a form or a gallery?
2. Which property makes a button go to another screen?
3. If you delete a gallery, what happens to the data in the list?
4. Which function saves an edit form to the data source?
5. Saving an app and publishing an app - what is the difference?

Answers: gallery | OnSelect | nothing, the data is untouched | SubmitForm | publishing releases it to users.

MODULES 14-18

Power Fx: Logic, Variables and Validation

The formula language that makes your app think

Power Fx is Excel for your app

In Excel

- =IF(A1 > 1000, "High", "Normal")
- Formulas recalculate automatically
- You reference cells
- No programming background needed

In Power Apps

- If(Value(txtAmount.Value) > 1000, "High", "Normal")
- Formulas recalculate automatically
- You reference controls and data
- Exactly the same way of thinking

If you have written an Excel formula, you have already written Power Fx.

The functions a business user actually needs

If()
Choose between outcomes

Filter()
Keep matching rows

Search()
Find text anywhere

Sort()
Order the results

LookUp()
Find one single record

Navigate()
Move between screens

Notify()
Show a message to the user

User()
Who is signed in

Today()
Current date

IsBlank()
Is this field empty?

Set()
Remember a value

Patch()
Write data directly

Twelve functions cover almost every business app. Learn these well before exploring anything else.

Variables: remembering something for later

Global variable - Set()

- Available on every screen
- Use for the selected record or user settings
- Set(varSelectedRequest, ThisItem)
- Lives until the app is closed

Context variable - UpdateContext()

- Available on one screen only
- Use for showing and hiding a panel
- UpdateContext({showPanel: true})
- Resets when you leave the screen

Think of a variable as a sticky note the app keeps for you until it needs the value again.

Remember the selected request

```
// Gallery OnSelect - store the row, then move on
Set(varSelectedRequest, ThisItem);
Navigate(RequestDetailsScreen, ScreenTransition.Fade)

// Details screen - display form Item property
varSelectedRequest

// Show a panel on this screen only
UpdateContext({showComments: true})
```

Check your work: the details screen must show the request you tapped, not the first one in the list.

Collections: a temporary table inside your app

A collection holds rows in the app's memory - useful for baskets, drafts and temporary selections.

```
// Add one item to a temporary shopping list
Collect(colBasket, {Item: txtItem.Value, Qty: Value(txtQty.Value)})

// Replace everything in the collection
ClearCollect(colBasket, {Item: "Laptop", Qty: 1})

// Empty it
Clear(colBasket)
```

A collection disappears when the app closes. Anything that must survive belongs in your list.

SubmitForm and Patch

SubmitForm()

- Works with a form control
- Handles validation and required fields
- The easiest way to save a record
- Best choice for the New Request screen

Patch()

- Works without a form control
- Updates specific fields only
- Useful for buttons like Approve or Cancel
- More flexible, slightly more to write

Rule of thumb: form on the screen means SubmitForm. No form, or updating one field, means Patch.

A simple Patch

```
// Create a new request without a form
Patch(
    'Employee Requests',
    Defaults('Employee Requests'),
    {
        EmployeeName: User().FullName,
        RequestType: drpType.Selected.Value,
        Status: "Pending"
    }
)
```

Defaults() means start with a blank new record. Swap it for an existing record to update instead.

Stop bad data before it is saved

Required fields

Description and Request Type must be filled in

Sensible numbers

Amount must be greater than zero

Realistic dates

Request date cannot be in the past

Clear feedback

Tell the user what to fix, politely and immediately

Validation is a business decision, not a technical one. Decide the rules with the process owner first.

Add validation to the request form

```
// Submit button OnSelect
If(
    IsBlank(txtDescription.Value),
    Notify("Please enter a description", NotificationType.Error),
    If(
        Value(txtAmount.Value) <= 0,
        Notify("Amount must be greater than zero", NotificationType.Warning),
        SubmitForm(frmRequest)
    )
)
```

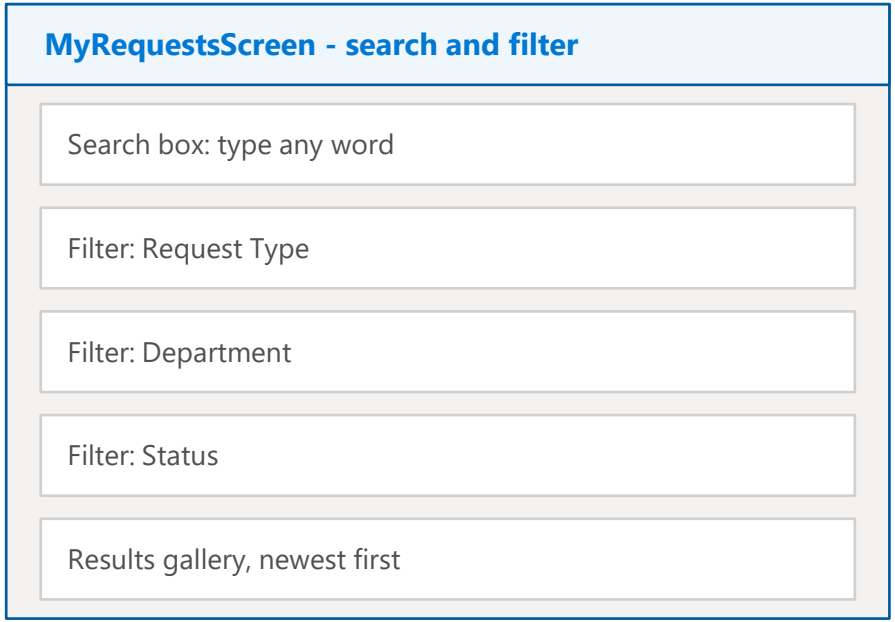
HANDS-ON: try to submit an empty form, read the message, then submit a valid one.

MODULES 19-22

Search, Status and Automated Approvals

Where the app becomes a real business process

Build a searchable request screen



Controls to add

- One Text input named txtSearch
- Three Dropdowns for type, department, status
- Include an All option in each dropdown
- Point the gallery's Items at the combined formula

Search, filter and sort together

```
SortByColumns(
    Filter(
        Search('Employee Requests', txtSearch.Value, "Title", "Description"),
        drpStatus.Selected.Value = "All" || Status.Value = drpStatus.Selected.Value
    ),
    "Created",
    SortOrder.Descending
)
```

Read it from the inside out: search the text, then keep the matching statuses, then sort newest first.

> The || symbol means 'or'. It lets the All option switch the filter off.

Status indicators at a glance

Pending

Awaiting the manager's decision

Approved

Decision made, request granted

Rejected

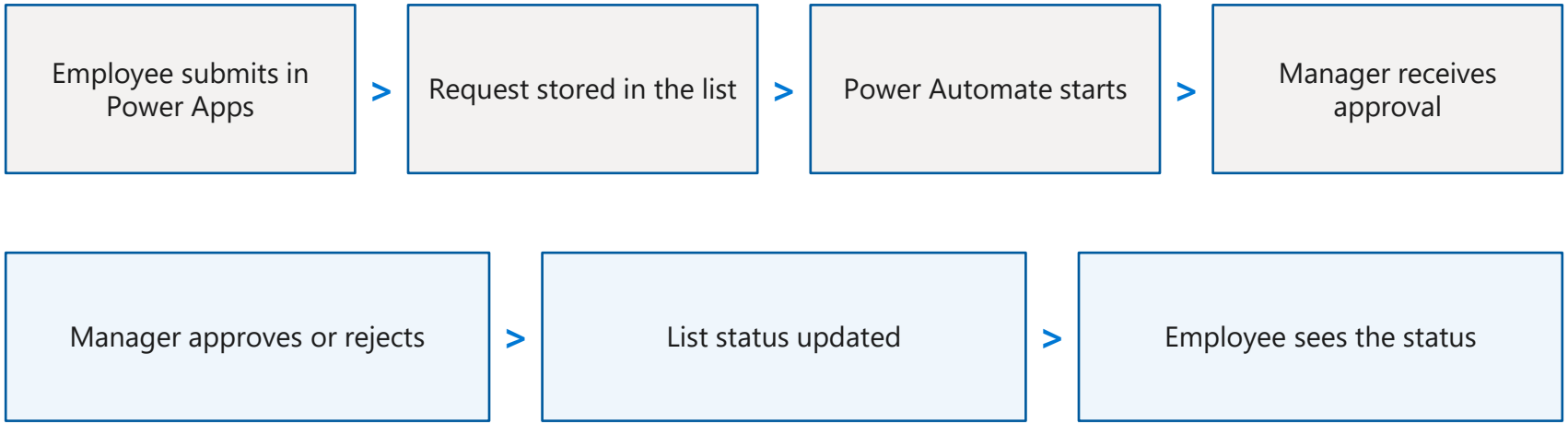
Declined, with a reason recorded

```
If(ThisItem.Status.Value = "Approved", Color.SeaGreen,
   If(ThisItem.Status.Value = "Rejected", Color.IndianRed, Color.DarkOrange))
```

Three colours, used consistently. Avoid turning a business app into a rainbow.

Introducing Power Automate

Power Automate does the chasing for you: it reacts to a new request and routes it to the right approver.

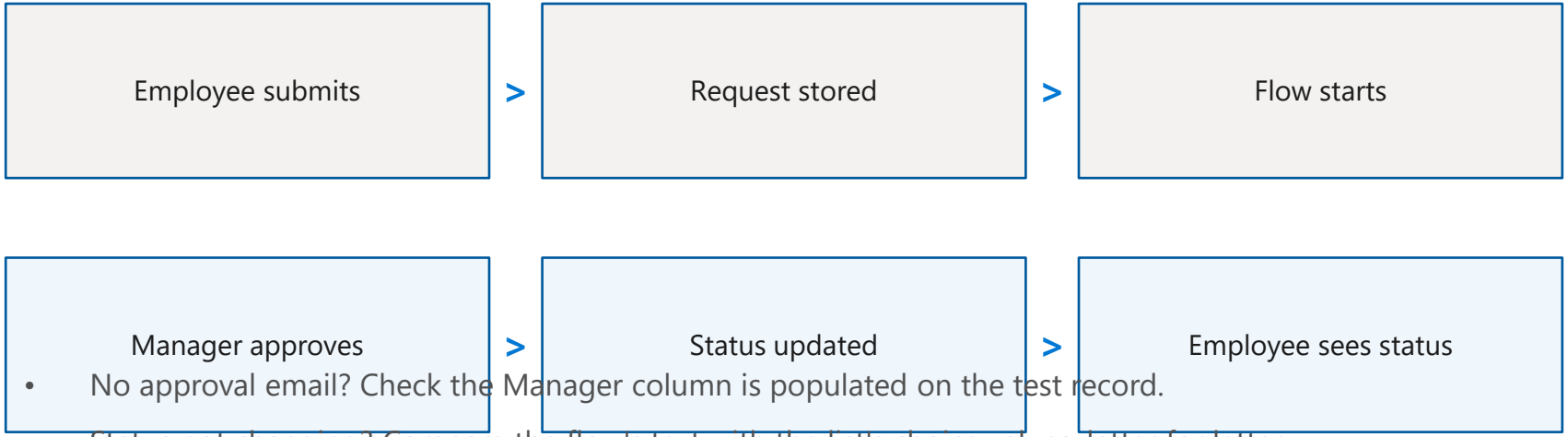


Build the Request Approval flow

1. Trigger: When an item is created - Employee Requests list.
2. Action: Start and wait for an approval - Approve or Reject, first to respond.
3. Assign it to the Manager field from the list item.
4. Condition: if the outcome equals Approve.
5. If yes - update the item, set Status to Approved. If no - set Status to Rejected.
6. Both branches: send an email to the requester with the manager's comments.

> Include the request details in the approval message so the manager can decide without opening the app.

Test the complete business process



- No approval email? Check the Manager column is populated on the test record.
- Status not changing? Compare the flow's text with the list's choice values, letter for letter.
- App still showing old data? Refresh the data source or reopen the screen.
- Flow failed? Open the run history - it shows exactly which step failed and why.

MODULES 23-27

Sharing, Mobile, AI and Collaboration

Getting the app safely into the hands of real users

Save, publish, share

Save

Stores your work in the studio. Users see nothing new yet.

Publish

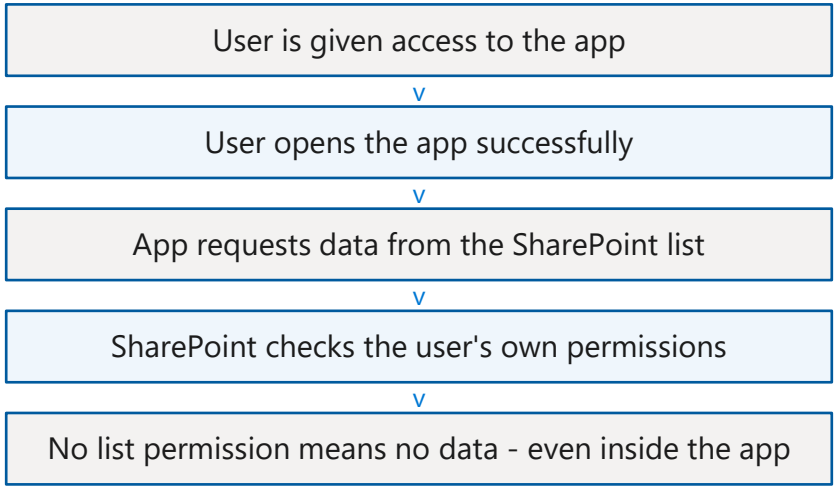
Releases the saved version. Users now get your changes.

Share

Grants people access to the app, individually or by group.

Three separate actions. Doing one does not do the others.

Sharing the app is not sharing the data



Governance checklist

- Grant list permissions alongside app access
- Use groups, not individual names
- Decide an owner for every app
- Know which environment the app lives in
- Review access when people change roles

Power Apps on mobile

Power Apps Mobile

One free app on iOS and Android holds all your business apps

Responsive interfaces

Containers keep screens usable on small displays

Camera

Attach a photo of the damaged equipment or the receipt

Location

Record where an inspection actually happened

Barcode scanning

Look up an asset by scanning its label

Offline awareness

Plan for weak signal in field scenarios

Mobile use cases: site inspection, asset tracking, attendance, field reporting.

AI in modern Power Apps

Describe to create

Explain the app you want in plain language and get a starting point

AI-assisted formulas

Describe the rule, review the suggested Power Fx

Copilot in the app

Let users ask questions about their own data

Microsoft 365 Copilot

Business apps surfaced where people already work

Availability depends on your licensing, environment, region, tenant settings and preview status - check before promising it.

How app building has evolved

Traditional development	Code > UI > Database > Logic
Low-code development	Data > Drag & drop > Power Fx > Automation
AI-assisted development	Describe requirement > AI-assisted design > App > Human refinement

Human judgement about the business process remains the part AI cannot supply.

Code view: awareness, not deep technique



Power Apps can show a control's configuration as structured, YAML-style text. It is useful for copying a control between apps, sharing an exact setup with a colleague, or reviewing what changed.

> You do not need to write code view by hand. Knowing it exists is enough for an end user.

Working as a team on one app

Coauthoring

More than one maker can work on the same app

Clear ownership

Name one owner accountable for the app

Naming standards

btnSubmit, galRequests, frmRequest - agreed and consistent

Change discipline

Test in preview, publish deliberately, tell users

Documentation

Record the data source, the flow and the approvers

Governance

Agree with IT which environment apps live in

Scenario: HR owns the app, Finance owns the approval rules, IT owns the environment. Agree this before you build.

CAPSTONE

Employee Request & Approval Management App

Bring every module together into one working solution

What your finished app must contain

Home screen

Welcome message, current user, navigation to New Request and My Requests.

New Request screen

Input form, validation rules, submit and cancel.

My Requests screen

Gallery with search, filters and status indicators.

Request Details screen

Full request information, status and manager comments.

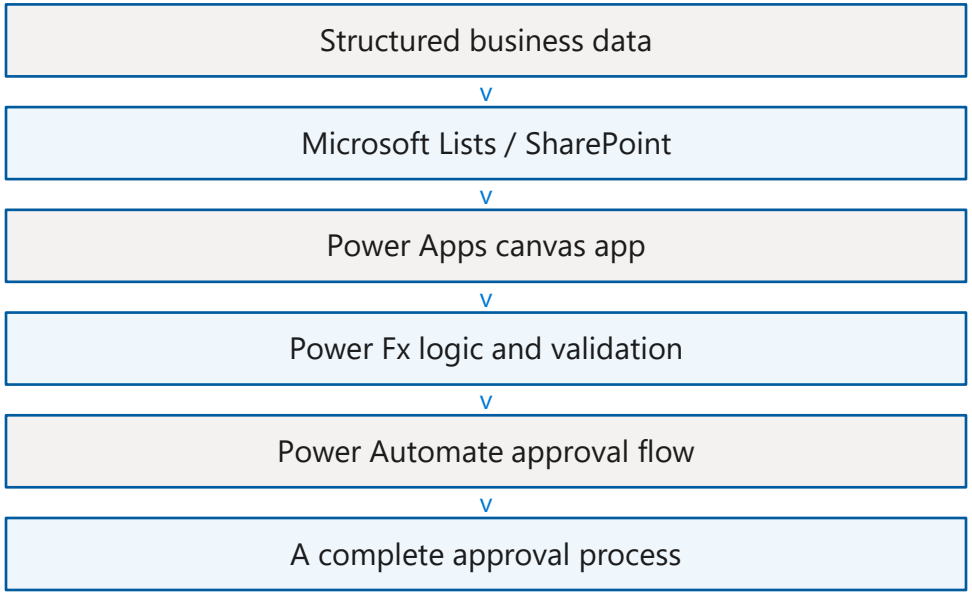
Plus automation: manager approval, decision recorded, status updated, requester notified.

Add three enhancements of your own

- **Search box**
- **Additional filters**
- **Status indicators**
- **Confirmation message**
- **Extra validation rule**
- **Mobile-friendly layout**
- **An additional screen**
- **Manager dashboard**
- **File attachments**

Choose any three, build them independently, then show the room what you made and why.

What you have built in two days



Power Fx cheat sheet

Navigate()
Go to a screen

Back()
Return

Filter()
Keep matching rows

Search()
Find text

Sort()
Order results

LookUp()
Find one record

If()
Choose an outcome

IsBlank()
Check if empty

Notify()
Message the user

User()
Who is signed in

Today()
Current date

SubmitForm()
Save a form

NewForm()
Blank form

ResetForm()
Discard edits

Patch()
Write data directly

Set()
Store a value

Print this page. These sixteen functions cover almost every app a business user will build.

Ten habits of a good app maker

- Plan your data before you design a single screen.
- Keep applications simple - solve one process well.
- Use meaningful control names from the very first control.
- Design for the person using it, not for the person building it.
- Use containers and responsive layouts.
- Validate user input before it reaches your data.
- Manage permissions on the data source, not just the app.
- Test with a real user before you publish.
- Avoid unnecessarily complex formulas - simple is maintainable.
- Automate the repetitive steps, keep the human decisions.

Where to go next

Power Apps beginner > **Advanced canvas apps** > **Power Automate**

Dataverse > **Model-driven apps** > **Copilot Studio and AI**



Practise first: rebuild the capstone for a process in your own department within the next two weeks. Skills fade quickly without a real problem to solve.

THANK YOU

You can build business apps without coding

Start with one process. Structure the data. Build the screen. Automate the approval.